

GIAO TIẾP QUA CỒNG WIFI

➤ Gợi ý mã nguồn rút gọn của chương trình điều khiển:

Khai báo các biến sử dụng

```
char TransmitData, ReceiveData;  
bit transmit flag, receive flag;
```

Khai báo các định nghĩa

```
#include <stdbool.h>  
#include <stdint.h>  
#include <stdio.h>  
#include <string.h>  
  
/** Some constants for ESP8266 **/  
#define ESP8266_STATION          0x01  
#define ESP8266_SOFTAP          0x02  
#define ESP8266_BOTH            0x03  
  
#define ESP8266_TCP              1  
#define ESP8266_UDP              0  
#define ESP8266_TRANS_PASS      1  
#define ESP8266_TRANS_NOR       0  
  
#define ESP8266_OK               1  
#define ESP8266_READY            2  
#define ESP8266_FAIL             3  
#define ESP8266_NOCHANGE         4  
#define ESP8266_LINKED           5  
#define ESP8266_UNLINK           6  
#define ESP8266_CONNECT          7
```

GIAO TIẾP QUA CỒNG WIFI

- **Gợi ý mã nguồn rút gọn của chương trình điều khiển:**

Đoạn chương trình xử lý ngắt

```
void interrupt(void)
{
    if((RCIE_bit == 1) && (RCIF_bit == 1))
    {
        RCIF_bit = 0;           // Clear interrupt bit
        ReceiveData = UART1_Read(); // Đọc dữ liệu từ UART
        receive_flag = 1;
    }
}
```

GIAO TIẾP QUA CỒNG WIFI

➤ Gợi ý mã nguồn rút gọn của chương trình điều khiển:

Đoạn chương trình con chức năng

```
/**Function to send one byte of data to UART**//  
void _esp8266_putch(char bt)  
{  
    while(!TXIF_bit); // hold the program till TX buffer is free  
    TXREG = bt; //Load the transmitter buffer with the received value  
}
```

```
/**Function to get one byte of data from UART**//  
char _esp8266_getch()  
{  
    if(OERR_bit) // check for Error  
    {  
        CREN_bit = 0; //If error -> Reset  
        CREN_bit = 1; //If error -> Reset  
    }  
    while(!RCIF_bit); // hold the program till RX buffer is free  
    return RCREG; //receive the value and send it to main function  
}
```

```
/**  
 * Output a string to the ESP module.  
 * This is a function for internal use only.  
 * @param ptr A pointer to the string to send.  
 */  
void _esp8266_print(unsigned char *ptr) {  
    while (*ptr != 0)  
    {  
        _esp8266_putch(*ptr++);  
    }  
}
```

GIAO TIẾP QUA CỒNG WIFI

- **Gợi ý mã nguồn rút gọn của chương trình điều khiển:**

```
/**
 * Wait until we found a string on the input.
 * Careful: this will read everything until that string (even if it's never
 * found). You may lose important data.
 * @param string
 * @return the number of characters read
 */
inline uint16_t _esp8266_waitFor(unsigned char *string)
{
    unsigned char so_far = 0;
    unsigned char received;
    uint16_t counter = 0;
    do
    {
        received = _esp8266_getch();
        counter++;
        if (received == string[so_far])
        {
            so_far++;
        }
        else
        {
            so_far = 0;
        }
    }
    while (string[so_far] != 0);
    return counter;
}
```

Đoạn chương trình con chức năng

GIAO TIẾP QUA CỒNG WIFI

- **Gợi ý mã nguồn rút gọn của chương trình điều khiển:**

```
/**
 * Restart the module
 * This sends the `AT+RST` command to the ESP and waits until there is a
 * response "OK" and "ready".
 */
void esp8266_restart(void)
{
    _esp8266_print("AT+RST\r\n");           // Command
    _esp8266_waitFor("OK");                 // Wait for "OK"
    _esp8266_waitFor("ready");              // Wait for "ready"
}
```

Đoạn chương trình con chức năng

```
/**
 * Check if the module is started
 * This sends the `AT` command to the ESP and waits until it gets a response "OK".
 */
void esp8266_isStarted(void)
{
    _esp8266_print("AT\r\n");              // Command
    _esp8266_waitFor("OK");                 // Wait for "OK"
}
```


GIAO TIẾP QUA CỒNG WIFI

➤ Gợi ý mã nguồn rút gọn của chương trình điều khiển:

Đoạn chương trình con chức năng

```
/**
 * Enable / disable command echoing.
 * Enabling this is useful for debugging: one could sniff the TX line from the
 * ESP8266 with his computer and thus receive both commands and responses.
 * This sends the ATE command to the ESP module and waits until it gets a response "OK".
 * @param echo whether to enable command echoing(1) or not(0)
 */
void esp8266_echoCmds(bool echo)
{
    _esp8266_print("ATE");           // Command
    if (echo)
    {
        _esp8266_putchar('1');
    }
    else
    {
        _esp8266_putchar('0');
    }
    _esp8266_print("\r\n");

    _esp8266_waitFor("OK");          // Wait for "OK"
}
```

GIAO TIẾP QUA CỒNG WIFI

➤ Gợi ý mã nguồn rút gọn của chương trình điều khiển:

```
/**
 * Set the WiFi mode.
 * ESP8266_STATION : Station mode
 * ESP8266_SOFTAP : Access Point mode
 * ESP8266_BOTH : Station + Access Point mode
 * This sends the AT+CWMODE command to the ESP module and waits until it gets a response "OK".
 * @param mode an ORed bitmask of ESP8266_STATION, ESP8266_SOFTAP, ESP8266_BOTH
 */
void esp8266_mode(unsigned char mode)
{
    _esp8266_print("AT+CWMODE=");    // Command
    _esp8266_putch(mode + '0');
    _esp8266_print("\r\n");
    _esp8266_waitFor("OK");          // Wait for "OK"
}
```

Đoạn chương trình con chức năng

```
/**
 * Set the transmission mode.
 * ESP8266_TRANS_PASS : Passthrough Receiving Mode (Transparent receiving transmission
 * ESP8266_TRANS_NOR : Normal Transmission Mode
 * This sends the AT+CIPMODE command to the ESP module and waits until it gets a response "OK".
 * @param mode an ORed bitmask of ESP8266_TRANS_PASS, ESP8266_TRANS_NOR
 */
void esp8266_trans_mode(unsigned char mode)
{
    _esp8266_print("AT+CIPMODE=");    // Command
    _esp8266_putch(mode + '0');
    _esp8266_print("\r\n");
    _esp8266_waitFor("OK");          // Wait for "OK"
}
```

GIAO TIẾP QUA CỒNG WIFI

- **Gợi ý mã nguồn rút gọn của chương trình điều khiển:**

Đoạn chương trình con chức năng

```
/**
 * Connect to an access point.
 * This sends the AT+CWJAP command to the ESP module and waits until it gets a response "OK".
 * @param ssid The SSID to connect to
 * @param pass The password of the network
 */
void esp8266_connect(unsigned char* ssid, unsigned char* pass)
{
    _esp8266_print("AT+CWJAP=\"");          // Command
    _esp8266_print(ssid);
    _esp8266_print("\",\"");
    _esp8266_print(pass);
    _esp8266_print("\r\n");
    _esp8266_waitFor("OK");                // Wait for "OK"
}
```


GIAO TIẾP QUA CỒNG WIFI

- **Gợi ý mã nguồn rút gọn của chương trình điều khiển:**

Đoạn chương trình con chức năng

```
/**
 * Open a TCP or UDP connection.
 * This sends the AT+CIPSTART command to the ESP module and waits until it gets a response "OK".
 * @param protocol Either ESP8266_TCP or ESP8266_UDP
 * @param ip The IP or hostname to connect to; as a string
 * @param port The port to connect to
 */
unsigned char esp8266_start(unsigned char protocol, unsigned char* ip, unsigned int port)
{
    unsigned char port_str[5] = "\0\0\0\0";
    _esp8266_print("AT+CIPSTART=\");
    if (protocol == ESP8266_TCP)
    {
        _esp8266_print("TCP");
    }
    else
    {
        _esp8266_print("UDP");
    }
    _esp8266_print("\",\");
    _esp8266_print(ip);
    _esp8266_print("\",");

    sprintf(port_str, "%u", port);
    _esp8266_print(port_str);
    _esp8266_print("\r\n");

    _esp8266_waitFor("OK");    // Wait for "OK"
}
```

GIAO TIẾP QUA CỒNG WIFI

- **Gợi ý mã nguồn rút gọn của chương trình điều khiển:**

Đoạn chương trình con chức năng

```
// Send data (AT+CIPSEND)
/**
 * Send data over a connection.
 * This sends the AT+CIPSEND command to the ESP module and waits until it gets a response "OK".
 * @param data The data to send
 */
void esp8266_send(void)    // For TCP Client Single Connection UART-Wifi Passthrough
{
    _esp8266_print("AT+CIPSEND");
    _esp8266_print("\r\n");
    _esp8266_waitFor("OK");
    while (_esp8266_getch() != '>');
}
```

GIAO TIẾP QUA CỒNG WIFI

- **Gợi ý mã nguồn rút gọn của chương trình điều khiển:**

```
/**
 * Stop sending data.
 * This sends the +++ to the ESP module.
 */
void esp8266_stop_send(void)
{
    _esp8266_print("+++");
    delay_ms(2000);
}
```

Đoạn chương trình con chức năng

```
/**
 * Delete TCP connection
 * This sends the AT+CIPCLOSE command to the ESP module.
 */
void esp8266_del_TCP(void)
{
    _esp8266_print("AT+CIPCLOSE\r\n");
    _esp8266_waitFor("OK");
}
```

GIAO TIẾP QUA CỒNG WIFI

- **Gợi ý mã nguồn rút gọn của chương trình điều khiển:**

```
void main(void)
{
    // Khai báo

    // Chương trình
    ADCON1 |= 0x0F;
    CMCON  |= 7;

    // Cấu hình Port B
    PORTB = 0x00; LATB = 0x00;
    TRISB0_bit = 1;

    // Cấu hình Port E
    PORTE = 0x00; LATE = 0x00;
    TRISE0_bit = 0;      // LED for Connected
    TRISE1_bit = 0;

    // Cấu hình Port C
    PORTC = 0x00; LATC = 0x00;
    TRISC0_bit = 0;      // Reset pin of ESP8266

    UART1_Init(115200);
    delay_ms(100);

    // Reset for ESP8266 module
    RCO_bit = 0; delay_ms(100); RCO_bit = 1;

    delay_ms(1000);      // For ESP8266 stable
}
```

Đoạn chương trình chính
Khai báo mặc định

GIAO TIẾP QUA CỒNG WIFI

- **Gợi ý mã nguồn rút gọn của chương trình điều khiển:**

Đoạn chương trình chính
Kích hoạt ngắt

```
RCIF_bit = 0;           // Clear interrupt bit
PIE1.RCIE = 1;          // Enable the EUSART receive interrupt

GIE_bit = 1;            // Enable global interrupt
PEIE_bit = 1;           // Enable peripheral interrupts
```


GIAO TIẾP QUA CỒNG WIFI

➤ Gợi ý mã nguồn rút gọn của chương trình điều khiển:

Đoạn chương trình chính
Cấu hình mô-đun ESP8266

```
// Configure: Station, TCP Client Single connection UART-Wifi Passthrough
esp8266_restart(); // Command: AT+RST - Restart module
esp8266_echoCmds(0); // Command: ATE0 - Enable(1)/Disable(0) echo
esp8266_isStarted(); // Command: AT - Wait till the ESP send back "OK"
esp8266_mode(ESP8266_STATION); // Command: AT+CMODE=1 - Put the module in Station mode
esp8266_connect("IOT_AI LAB","rC7YNUqM"); // Command: AT+CWJAP="ssid","pass" - Connect to an access point
esp8266_start(ESP8266_TCP, "192.168.1.49", 8000); // Command: AT+CIPSTART="protocol","ip",port - Start a TCP network in that IP and port
RE0_bit = 1; // Warning - Connect to Server (LED Connect ON)
esp8266_trans_mode(ESP8266_TRANS_PASS); // Command: AT+CIPMODE="mode" - Select Passthrough Receiving Mode (Transparent Receiving Transmission)
esp8266_send(); // Command: AT+CIPSEND - Send data
```

Protocol: **TCP Client**

GIAO TIẾP QUA CỒNG WIFI

➤ Gợi ý mã nguồn rút gọn của chương trình điều khiển:

Đoạn chương trình chính
Xử lý điều khiển và thu thập

```
while(1)
{
    /***** Button 1 *****/
    if (Button(&PORTB, 0, 10, 0))
    {
        while(Button(&PORTB, 0, 10, 0));
        TransmitData = 'S';
        UART1_Write(TransmitData);
    }

    if(receive_flag == 1)
    {
        receive_flag = 0;
        ReceiveData = UART1_Read();
        if (ReceiveData == '@')
        {
            LATE1_bit = 1;
            TransmitData = 'O';
            UART1_Write(TransmitData);
        }
        else if (ReceiveData == '$')
        {
            LATE1_bit = 0;
            TransmitData = 'F';
            UART1_Write(TransmitData);
        }
        else if (ReceiveData == 'Z')    // Ngat ket noi
        {
            GIE_bit = 0;                // Disable global interrupt
            RE0_bit = 0;                // Warning: Disconnect Server (LED Connect OFF)

            esp8266_stop_send();        // Command: +++ - Stop sending data
            esp8266_trans_mode(ESP8266_TRANS_NOR); // Command: AT+CIPMODE="mode" - Disable UART - WiFi passthrough mode (transparent transmission)
            esp8266_del_TCP();          // Command: AT+CIPCLOSE - Delete TCP connection
        }
    }
}
```

GIAO TIẾP QUA CỒNG WIFI

- Gợi ý mã nguồn rút gọn của giao diện mẫu:



Minh họa giao diện điều khiển trên máy tính để giao tiếp với thiết bị ngoại vi thông qua cổng Wifi

➤ Gợi ý mã nguồn rút gọn của giao diện mẫu:

Khai báo thư viện sử dụng

```
using System;  
using System.Collections.Generic;  
using System.ComponentModel;  
using System.Data;  
using System.Drawing;  
using System.Linq;  
using System.Text;  
using System.Threading.Tasks;  
using System.Windows.Forms;  
using System.Net;  
using System.Net.Sockets;  
using System.Threading;
```

GIAO TIẾP QUA CỒNG WIFI

phamquangtri_dt@iuh.edu.vn

➤ Gợi ý mã nguồn rút gọn của giao diện mẫu:

Khai báo các biến sử dụng

```
IPEndPoint ipe;  
Socket server;  
Socket client;  
byte[] datasend = new byte[1];           // Mang gửi đi  
byte[] datareceive = new byte[1];        // Mang nhận về  
int count = 0;
```

```
public Form_Main()  
{  
    InitializeComponent();  
    Control.CheckForIllegalCrossThreadCalls = false;  
}
```

Đoạn chương trình chính khởi động

➤ Gợi ý mã nguồn rút gọn của giao diện mẫu:

```
private void button_Connect_Click(object sender, EventArgs e)
{
    Thread thread = new Thread(Endpoint_Thread);
    thread.IsBackground = true;
    thread.Start();

    textBox_Status.BackColor = Color.Yellow;        // Hieu chỉnh mau va thong tin
    textBox_Status.Text = "Waiting for device to connect";

    // Khoa IP va Port
    textBox_ServerIP_1.Enabled = false;
    textBox_ServerIP_2.Enabled = false;
    textBox_ServerIP_3.Enabled = false;
    textBox_ServerIP_4.Enabled = false;
    textBox_ServerPort.Enabled = false;
}
```

Đoạn chương trình tạo kết nối Server

Protocol: **TCP Server**

➤ Gợi ý mã nguồn rút gọn của giao diện mẫu:

```
void Endpoint_Thread()
{
    try
    {
        // Thiết lập IP Endpoint và Socket
        string ip = textBox_ServerIP_1.Text.Trim() + "." + textBox_ServerIP_2.Text.Trim() + "." + textBox_ServerIP_3.Text.Trim() + "." + textBox_ServerIP_4.Text.Trim();
        int port = int.Parse(textBox_ServerPort.Text.Trim());
        IPEndPoint ipe = new IPEndPoint(IPAddress.Parse(ip), port);
        server = new Socket(AddressFamily.InterNetwork, SocketType.Stream, ProtocolType.Tcp);

        // Kết nối Socket với IP Endpoint của máy chủ, kết nối
        server.Bind(ipe);

        // Lắng nghe từ Client
        server.Listen(10); // Cho phép kết nối tối đa 10 Client

        // Chấp nhận kết nối
        client = server.Accept();
        textBox_Status.BackColor = Color.Lime; // Hiệu chỉnh màu và thông tin
        textBox_Status.Text = "Connected with: " + client.RemoteEndPoint.ToString();

        Thread thread = new Thread(Receive);
        thread.IsBackground = true;
        thread.Start();

        // Cho phép các nút điều khiển hoạt động
        button_Connect.Enabled = false;
        button_DisConnect.Enabled = true;
        button_ON.Enabled = true;
        button_OFF.Enabled = true;
    }
    catch (Exception)
    {
        MessageBox.Show("Check the connection again.", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);

        textBox_Status.BackColor = Color.Red; // Hiệu chỉnh màu và thông tin
        textBox_Status.Text = "Not connect";

        // Mù IP và Port
        textBox_ServerIP_1.Enabled = true;
        textBox_ServerIP_2.Enabled = true;
        textBox_ServerIP_3.Enabled = true;
        textBox_ServerIP_4.Enabled = true;
        textBox_ServerPort.Enabled = true;

        // Các nút điều khiển hoạt động
        button_Connect.Enabled = true;
        button_DisConnect.Enabled = false;
        button_ON.Enabled = false;
        button_OFF.Enabled = false;
    }
}
```

Đoạn chương trình tạo kết nối Server

Protocol: **TCP Server**

➤ Gợi ý mã nguồn rút gọn của giao diện mẫu:

```
private void button_DisConnect_Click(object sender, EventArgs e)
{
    try
    {
        datasend = Encoding.ASCII.GetBytes("Z"); // Gửi mã ngắt kết nối
        client.Send(datasend, datasend.Length, SocketFlags.None);

        server.Close();
        client.Close();
    }
    catch (Exception)
    {
        MessageBox.Show("Check the connection again.", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }

    textBox_Status.BackColor = Color.Red; // Hiệu chỉnh màu và thông tin
    textBox_Status.Text = "Not connect";

    // Mở IP và Port
    textBox_ServerIP_1.Enabled = true;
    textBox_ServerIP_2.Enabled = true;
    textBox_ServerIP_3.Enabled = true;
    textBox_ServerIP_4.Enabled = true;
    textBox_ServerPort.Enabled = true;

    // Cam các nút điều khiển hoạt động
    button_Connect.Enabled = true;
    button_DisConnect.Enabled = false;
    button_ON.Enabled = false;
    button_OFF.Enabled = false;
}
```

Đoạn chương trình ngắt kết nối Server

Protocol: **TCP Server**

➤ Gợi ý mã nguồn rút gọn của giao diện mẫu:

```
private void Form_Main_FormClosing(object sender, FormClosingEventArgs e)
{
    DialogResult answer = MessageBox.Show("Do you want to exit the program?", "Question", MessageBoxButtons.YesNo, MessageBoxIcon.Question);
    if (answer == DialogResult.No)
    {
        e.Cancel = true;
    }
    else
    {
        if (button_DisConnect.Enabled == true)
        {
            server.Close();
            client.Close();
        }
    }
}
```

Đoạn chương trình đóng giao diện

➤ Gợi ý mã nguồn rút gọn của giao diện mẫu:

```
private void Receive()
{
    try
    {
        while (true)
        {
            int temp = client.Receive(datareceive);
            string s = Encoding.ASCII.GetString(datareceive, 0, temp);
            if (s == "0")
            {
                pictureBox_LED.Image = MySample.Properties.Resources.LED_RED;
            }
            else if (s == "F")
            {
                pictureBox_LED.Image = MySample.Properties.Resources.LED_OFF;
            }
            else if (s == "S")
            {
                count++;
                textBox_SW.Text = count.ToString();
            }
        }
    }
    catch (Exception)
    {
        client.Close();
    }
}
```

Đoạn chương trình thu thập dữ liệu

➤ Gợi ý mã nguồn rút gọn của giao diện mẫu:

```
private void button_ON_Click(object sender, EventArgs e)
{
    try
    {
        datasend = Encoding.ASCII.GetBytes("@");
        client.Send(datasend, datasend.Length, SocketFlags.None);
        //client.SendTo(datasend, datasend.Length, SocketFlags.None, client.RemoteEndPoint);
    }
    catch (Exception)
    {
        MessageBox.Show("Check the connection again.", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}
```

Đoạn chương trình điều khiển LED bật

```
private void button_OFF_Click(object sender, EventArgs e)
{
    try
    {
        datasend = Encoding.ASCII.GetBytes("$");
        client.Send(datasend, datasend.Length, SocketFlags.None);
        //client.SendTo(datasend, datasend.Length, SocketFlags.None, client.RemoteEndPoint);
    }
    catch (Exception)
    {
        MessageBox.Show("Check the connection again.", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}
```

Đoạn chương trình điều khiển LED tắt

➤ Gợi ý mã nguồn rút gọn của giao diện mẫu:

```
private void textBox_ServerIP_1_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar) && !char.IsDigit(e.KeyChar))
    {
        e.Handled = true;
    }
}
```

Đoạn chương trình kiểm tra hợp lệ
(chỉ nhận giá trị số)

```
private void textBox_ServerIP_1_Validated(object sender, EventArgs e)
{
    if (textBox_ServerIP_1.Text == "")
    {
        MessageBox.Show("Blank is not a valid entry. Please specify a value between 1 and 223", "Error", MessageBoxButtons.OK, MessageBoxIcon.Exclamation);
        textBox_ServerIP_1.Text = "192"; textBox_ServerIP_1.Focus();
    }
    else if (Int16.Parse(textBox_ServerIP_1.Text) < 1)
    {
        MessageBox.Show(textBox_ServerIP_1.Text + " is not a valid entry. Please specify a value between 1 and 223", "Error", MessageBoxButtons.OK, MessageBoxIcon.Exclamation);
        textBox_ServerIP_1.Text = "192"; textBox_ServerIP_1.Focus();
    }
    else if (Int16.Parse(textBox_ServerIP_1.Text) > 223)
    {
        MessageBox.Show(textBox_ServerIP_1.Text + " is not a valid entry. Please specify a value between 1 and 223", "Error", MessageBoxButtons.OK, MessageBoxIcon.Exclamation);
        textBox_ServerIP_1.Text = "192"; textBox_ServerIP_1.Focus();
    }
}
```

Đoạn chương trình kiểm tra hợp lệ
(giới hạn giá trị và không bỏ trống)